Université Bretagne Sud
IUT de Vannes
Département Informatique — 2026/2027

BUT 2 — R3.07. SQL dans un langage de programmation

TD/TP 5

Patron Repository (suite JDBC et JPA)

Contexte

Lors des TP précédents, vous avez développé une application bancaire en plusieurs étapes :

- accès à la base de données via **JDBC** (DAO, requêtes SQL explicites) ;
- migration vers **JPA** avec des entités persistantes et un `EntityManager`.

Bien que JPA simplifie fortement l'accès aux données, le code métier reste encore **couplé à l'API JPA** (EntityManager, transactions, requêtes).

Dans ce TP, vous allez introduire le **patron Repository**, dont l'objectif est de :

- encapsuler totalement l'accès aux données ;
- isoler la logique métier de la technologie de persistance ;
- améliorer la testabilité et l'évolutivité de l'application.

Dans les frameworks modernes tels que **Spring Data JPA**, l'accès aux données repose sur le **patron Repository**. Avec Spring Data :

- le développeur définit uniquement des **interfaces Repository** ;
- les méthodes CRUD sont fournies automatiquement ;
- la logique métier ne dépend plus ni de SQL, ni de JPA.

Cependant, derrière cette automatisation, Spring Data repose sur une implémentation concrète du patron Repository, historiquement implémentée **manuellement** au-dessus de JPA.

Dans ce TP, vous allez implémenter **manuellement un Repository en JPA**, afin de comprendre les principes fondamentaux du patron Repository, avant son utilisation automatisée dans Spring Data.

1 Différence DAO et Repository

Dans les pratiques modernes, le patron Repository remplace le DAO (*Data Access Object*) mais il le généralise et l'élève à un niveau d'abstraction plus métier.

Le **DAO** est né avec JDBC. Il est fortement lié au SQL, à la base de données et à la structure des tables. On définit dans une classe DAO des opérations souvent techniques :

```
findClientById(int id)
insertClient(Client c)
updateClient(Client c)
```

Une classe DAO représente une table.

Le **Repository** est issu de la **conception pilotée par le domaine**[1] (*Domain-Driven Design* – DDD). Une interface Repository représente une abstraction métier, pas technique. Elle manipule des agrégats métier. Elle cache complètement JPA, SQL et la base de données. Le Repository représente une collection d'objets métier.

Dans certains cas rares, DAO et Repository coexistent. Des classes DAO contiennent alors des accès SQL très spécifiques pour des requêtes complexes ou pour viser un haut niveau de performance.

Objectifs pédagogiques

À l'issue de ce TP, vous devrez être capables de :

- comprendre le rôle du patron Repository ;
- distinguer DAO, Repository et Service ;
- concevoir une interface Repository indépendante de JPA ;
- implémenter un Repository basé sur JPA ;
- utiliser le Repository dans une couche service.

Exercice – Mise en place du patron Repository

On considère les entités JPA déjà développées lors du TP précédent, notamment :

- `Client`
- `Compte`

Les opérations courantes sur les clients sont :

- création d'un client ;
- recherche d'un client par identifiant ;
- récupération de tous les comptes d'un client.

Travail demandé

Le code à l'intérieur des méthodes peut être généré par IA. Mais la structure globale du projet et les différentes interface et classes, ainsi que leurs dépendances doivent être créées à la main.

a) Définir une interface `ClientRepository` contenant les méthodes suivantes :
   - `save(Client client);`
   - `findById(int id);`
   - `findAll();`
   - `findComptesByClient(int idClient).`

b) Implémenter cette interface dans une classe `ClientRepositoryJPA` utilisant un `EntityManager`.

c) Créer une classe `BanqueService` qui :
   - dépend uniquement de l'interface `ClientRepository` ;
   - ne manipule jamais directement l'EntityManager.

d) Illustrer l'utilisation du service dans une classe `Main` :
   - création d'un client ;
   - récupération de ses comptes.

e) Expliquer en quelques lignes :

1. https://martinfowler.com/bliki/DomainDrivenDesign.html

— la différence entre DAO et Repository ;
— les avantages du Repository par rapport à l'approche JPA directe.

Contraintes

— Les entités JPA ne doivent pas dépendre du Repository.
— Le service métier ne doit pas dépendre de JPA.
— Toute opération de persistance doit passer par un Repository.

Rendu attendu

— Interfaces et implémentations du Repository ;
— Classe service utilisant le Repository ;
— Code clair et commenté ;
— Réponses argumentées aux questions.